Exact Resource Thresholds and Information Limits in a Unit-Latency Reorder-Buffer Model

Research draft, 14 October 2026 (Asia/Singapore)

ILP1-20261014. Correctness and historical novelty are separate questions.
No author list, publication license, submission, or DOI is determined by this draft.

Abstract

We study offline and online unit-job scheduling with eager in-order admission, bounded unretired occupancy, and in-order retirement. We prove an exact execution-time reduction to precedence scheduling by adding every edge from index i to index j with $i + B \leq j$. This is a specialized formal bridge to established resource-event graph methods, not a claim to have invented them. We give exact algorithms and independently checked certificates, and exhibit two-input programs whose minimum sufficient window falls from N to 3 when issue width increases from 2 to 3. This unbounded threshold change costs only one extra cycle at window two, highlighting the distinction between exact and approximate saturation. A separate chain-prefix family has linear window requirements even at fixed positive relative tolerance. Under a precise hidden-future information model, a universally optimal online policy exists exactly when $W = 1$ or $W \geq B$. Lean 4.34.1 verifies the operational reduction, existence of an optimum, an executable exact enumeration algorithm, resource caps, Bellman certificates, and a four-job arbitrary-policy online obstruction. General resource families and online classifications have independent written review but are not yet fully formalized. Historical originality remains unconfirmed.

1 Introduction

For a fixed program, increasing processor resources cannot worsen its optimal duration. This does not imply that the least amount of one resource needed to attain an optimum changes monotonically with another resource. Nor does the existence of an offline optimum imply that a finite-information scheduler can always select it. We separate these questions in a deliberately narrow model.

The main structural examples concern exact resource thresholds, rather than real hardware speedups. A second result classifies the resource regimes in which hidden future dependencies obstruct universal online optimality. The foundational reduction and algorithms are included to make these claims precise, reproducible, and amenable to formal checking. The result catalogue and complete audit records accompany this draft in the research workstream.

2 Formal model

The program consists of N fixed operations with indices $0, \ldots, N-1$. Its true dependency graph E has only forward edges $i < j$. Operations are deterministic, have unit latency, and execute exactly once. Two-input, one-output operations suffice for the principal examples. SSA or versioned registers remove naming dependencies; this assumes pure operations and preserves producer values. Instruction elimination and algebraic rewriting are excluded; an isolated operation may also produce an observable output.

At a cycle boundary let D be the set of completed operations and let r be the least incomplete index, or N at termination. With $B, W \geq 1$, the eager window is $[r, \min(N, r + B))$. Completed entries behind r remain in the window until earlier operations finish. In one cycle any subset of at most W ready, incomplete window entries may issue. A predecessor must have completed in a previous cycle. Issued jobs complete at the cycle end; the whole completed prefix retires and the next cycle refills immediately. There is no same-cycle dependent bypass.

A schedule is a map $t : \{0, \ldots, N-1\} \to \mathbb{N}$ of issue times. For nonempty programs its duration is $1 + \max_j t(j)$; empty programs take zero cycles and have no defined IPC. Let $T^*(E, B, W)$ be the minimum duration and $IPC^* = N/T^*$. Both resources may be capped at N for nonempty inputs; effective width is at most B.

An offline planner sees the entire graph. An online planner sees only admitted operations, their incoming true dependencies, and execution history. It may know N, but receives no oracle for future uses of visible values. All online lower bounds below use this observation model.

3 Related work and claim boundaries

Wall's report studies instruction-level parallelism under idealization [1]; Dubey, Adams, and Flynn analyze instruction windows [2]. The latter full text was not obtained in this audit. Tullsen and Calder already add window-edge dependencies to model ordered window admission in a 1998 report [4]. Fields, Rubin, and Bodík use execution-event DAGs with resource edges [3]; later work explicitly records FIFO/ROB commit-to-dispatch constraints [5]. Our all-prefix execution-only reduction must be read as a specialization of this prior modeling tradition.

Bellman recursion, finite exhaustive search, precedence longest paths and Graham's list-scheduling guarantee are classical. At width two the reduced problem is covered by Coffman–Graham's polynomial algorithm [6, 7]. We do not claim a new solution of unrestricted precedence scheduling. A September 2026 OpenAI artifact claims a polynomial three-machine result with Lean sources [8]; its actual endpoint was inspected but its full dependency closure was not independently compiled here. Neither unconditional acceptance nor a statement that this question simply remains open is justified by our current audit.

Hidden-successor adversaries and advice-counting methods also have prior art; the workstream literature notes compare different release and observation models. No exact equivalent of all the threshold formulas below was located, but the search is incomplete. All prospective novelty is unconfirmed.

4 Static reduction and exact algorithms

Theorem 1 (Exact window representation)**.** *Define*

$$E_B = E \cup \{(i, j) : i + B \leq j < N\}.$$

A time assignment is legal in the ROB model if and only if it strictly respects E_B and every time layer has at most W jobs.

Proof. When j issues, every index $i \leq j - B$ must already have retired, hence completed in a previous cycle. This gives all the artificial precedences; true data and issue-width conditions are necessary as well.

Conversely replay the time layers. Before $t(j)$ all indices $i \leq j - B$ have completed. Thus the retirement frontier satisfies $r > j - B$ when $j \geq B$, and admission is immediate when $j < B$. Since j is still incomplete, $r \leq j < r + B$. It is in the window, all true predecessors are ready, and the proposed layer fits the issue width. Induction over time proves legality, including schedules with idle cycles. □

Only adding the offset edge $(j-B, j)$ is insufficient: for $N=4, B=2$ and $E=\varnothing$, the times $(1,0,2,1)$ satisfy both offset edges but violate the required edge $0 \to 3$.

Proposition 2 (Exact search and certificates)**.** *An optimal finite schedule exists. If $A(D)$ is the ready window set, its value satisfies*

$$V(D) = 1 + \min_{\varnothing \neq S \subseteq A(D),\ |S| \leq W} V(D \cup S), \qquad V(\{0, \ldots, N-1\}) = 0.$$

A legal K-cycle witness plus a potential v on every reachable state with $v(\varnothing) = K$, $v(\text{all}) = 0$ and $v(D) \leq 1 + v(D \cup S)$ for every legal transition proves optimality.

Proof. The earliest incomplete operation is ready because all its true predecessors have smaller indices. Issuing it repeatedly finishes within N cycles. Idle steps are unnecessary, and each nonempty step strictly increases $|D|$. Induction on $N - |D|$ proves the recurrence and constructs its attaining schedule. Telescoping the potential inequality along any execution gives the lower bound K; the legal witness gives the opposite bound. □

The independent checker enumerates successors itself and rejects omitted reachable states. This is a full-state certificate, not a compact one. There are at most $1 + N2^{B-1}$ completed-set states. Labeling each frontier position completed, selected, or unselected bounds aggregate transitions by $O(N3^B)$ before polynomial feasibility-checking factors.

For $W \geq B$, any E_B antichain lies in fewer than B index gaps and has at most B vertices. Earliest precedence times are therefore feasible and $T^*(E, B, W) = h(E_B)$, where height counts vertices. The recurrence

$$e(j) = 1 + \max\left(\max_{(i,j) \in E} e(i),\ \max_{i \leq j-B} e(i) \right),$$

with empty maximum -1, is evaluated in $O(N + |E|)$ using prefix maxima. For $W \geq N$, binary search on B computes B_{sat} in $O((N + |E|) \log N)$. A height-optimal schedule, a true critical path, and a longer augmented path at $B-1$ (only when $B > 1$) form an $O(N)$ certificate of its minimality. At $B = 1$ no smaller positive window needs exclusion.

For any fixed true-dependency-respecting schedule, define

$$\beta(t) = \max(\{1\} \cup \{j - i + 1 : i < j,\ t(i) \geq t(j)\}), \quad \omega(t) = \max_c |t^{-1}(c)|.$$

The reduction gives legality exactly when $B \geq \beta(t)$ and $W \geq \omega(t)$. Minimizing these footprints over the appropriate optimal schedules characterizes saturation, but is not itself an efficient general algorithm.

5 Unbounded changes in exact resource thresholds

For nonempty programs define

$$B_{\text{sat}}(E, W) = \min\{B : T^*(E, B, W) = T^*(E, N, W)\},$$
$$W_{\text{sat}}(E, B) = \min\{W : T^*(E, B, W) = T^*(E, B, B)\}.$$

Optimal duration is nonincreasing in each resource, since more window space removes artificial edges and more issue capacity relaxes layer size. The threshold functions do not inherit that monotonicity.

Theorem 3 (A fixed-width window collapse)**.** *For every integer $K \geq 2$ there is a program P_K of $N = 2K$ operations, with true indegree and outdegree at most two, such that*

$$B_{\text{sat}}(P_K, 2) = 2K, \qquad B_{\text{sat}}(P_K, 3) = 3.$$

Both unrestricted-window optima equal K.

Proof. Job 0 is isolated. The other jobs are pair layers $\{1,2\},\{3,4\},\ldots,\{2K-3,2K-2\}$ followed by singleton $\{2K-1\}$. Between successive layers add all forward edges. Every nonisolated job lies on a K-vertex path. Consequently any K-cycle schedule forces the job in layer ℓ into cycle $\ell-1$.

At width two every pair layer fills its cycle; job 0 must share the final cycle with job $2K-1$. This is legal at $B=N$ and meets the height bound. At every $B<N$, however, the artificial edge $0\to 2K-1$ prohibits that common time. Hence $B_{\mathrm{sat}}(P_K,2)=N$.

At $B=W=3$, issue $\{0,1,2\}$ first, then successive pairs, then the final singleton. Every batch is admitted and the duration is K. At $B\le 2$, job 2 is not admitted initially, contradicting its forced initial time in every K-cycle schedule. Thus the second threshold is exactly three. □

The additive threshold drop $2K-3$ and the ratio $2K/3$ are unbounded, even for fixed issue widths. This does *not* imply an unbounded performance gain. At width two and every $2\le B<2K$, the exact duration is only $K+1$: issue $\{0,1\}$, then $\{2\}$, then the remaining pair layers and the final singleton. Window two already suffices for this schedule, and the theorem excludes K cycles. At window one the duration is $2K$.

Define $B_\varepsilon(E,W)$ by allowing duration at most $(1+\varepsilon)T^*(E,N,W)$. The exact curves for this family are

	$0\le\varepsilon<1/K$	$1/K\le\varepsilon<1$	$\varepsilon\ge 1$
$W=2$	$2K$	2	1
$W=3$	3	2	1

Thus this example's huge exact threshold is fragile under fixed relative tolerance as K grows. The following separate construction shows that such fragility is not universal.

Theorem 4 (A depth-two width-saturation curve)**.** *Let $m\ge 2$ and $1\le r\le\lfloor m/2\rfloor$. Take r initial isolated roots, then m required roots, then $m-r$ sinks. Partition the required roots into r pairs and $m-2r$ singletons, assigning each group to a different sink. For $1\le B\le 2m$,*

$$T^*(E,B,W)\le 2 \quad\Longleftrightarrow\quad B\ge m+r \ \text{ and } \ W\ge m+\min(r,2m-B).$$

In particular, $W_{\mathrm{sat}}(E,B)=m+\min(r,2m-B)$ for $m+r\le B\le 2m$.

Proof. All m required roots must issue first, so the last one must be initially admitted: $B\ge m+r$. Admission of the final job in the next cycle forces the first $2m-B$ jobs to complete initially. The union of these jobs and the required roots has size $m+\min(r,2m-B)$, giving the width bound. Conversely issue exactly that union first. Its prefix retirement admits the whole remaining program, every sink is ready, and at most m jobs remain for the second cycle. Any true edge excludes a one-cycle schedule. □

As B grows from $2m-r$ to $2m$, the sufficient width decreases by r. Original indegree is at most two and outdegree at most one.

Theorem 5 (Robust linear window demand)**.** *Let Q_K consist of a chain in indices $0,\ldots,K-1$, followed by K independent operations, where $K\ge 1$. For all $B\ge 1, W\ge 2$,*

$$T^*(Q_K,B,W)=K+\left\lceil\frac{\max(0,K-B+1)}{\min(B,W)}\right\rceil.$$

Writing $q=\lfloor\varepsilon K\rfloor$ for $\varepsilon\ge 0$, its exact approximate saturation threshold is

$$B_\varepsilon(Q_K,W)=\max\left(\left\lceil\frac{K+1}{q+1}\right\rceil,\ K+1-qW\right).$$

For every fixed $W\ge 2$ and $0\le\varepsilon<1/W$, this threshold is $\Omega(N)$, although true indegree and outdegree are at most one.

Proof. The final chain operation completes no earlier than the end of the Kth cycle. At the beginning of its issue cycle, the retirement frontier is at most $K-1$. Hence at most $B-1$ independent operations have even been admitted, including any issued alongside this last chain operation. At least $Q = \max(0, K-B+1)$ independent operations remain afterward, requiring $\lceil Q/\min(B,W) \rceil$ further cycles. Delaying the chain cannot evade this admission bound.

For attainment, run the chain without gaps. In each of its last $\min(K, B-1)$ cycles also issue the next independent operation. These operations are admitted as the chain prefix retires, and width at least two suffices. After the chain finishes, the remaining Q independent operations can be issued in consecutive batches of size $\min(B,W)$. This proves the duration formula, including $B=1$ and $B>N$.

At unrestricted window the optimum is K, so the allowed extra cycles are q. The condition for a window B is exactly $K+1 \le B + q\min(B,W)$. The identity

$$B + q\min(B,W) = \min((q+1)B,\ B+qW)$$

turns this into the two lower bounds in the formula. In particular $B_\varepsilon \ge K+1-W\lfloor \varepsilon K \rfloor \ge (1-W\varepsilon)K+1$, proving the final claim. □

For width two and $0 \le \varepsilon < 1$, this simplifies to $\max(2, K+1-2\lfloor \varepsilon K \rfloor)$; at $\varepsilon \ge 1$ it is one. At width one, the unrestricted baseline is instead $2K$, and the relative saturation window is one for every nonnegative tolerance.

Corollary 6 (A robust single-slot trade-off)**.** *Fix an integer $W \ge 2$ and $1/(W+1) \le \varepsilon < 1/W$. On the same family, $B_\varepsilon(Q_K, W) = \Omega(N)$ whereas $B_\varepsilon(Q_K, W+1) \le W+1$. Thus increasing issue width by one can reduce the approximate sufficient window by an unbounded factor at fixed positive tolerance.*

Proof. The lower bound follows from the theorem. Since $q \ge \lfloor K/(W+1) \rfloor$, integer division gives $K+1 \le (W+1)(q+1)$. Both terms in the formula for width $W+1$ are therefore at most $W+1$. □

For width two this applies to every fixed $1/3 \le \varepsilon < 1/2$. This concerns approximate thresholds: the exact threshold of Q_K is $K+1$ for all $W \ge 2$. Program order matters. Interleaving the same chain with its independent operations allows $B = W = 2$ to finish in K cycles, but reordering is excluded here. The relevant object is a DAG with its fixed topological program order, not an unlabeled partial order alone.

6 Information limits for online scheduling

Theorem 7 (Universal-optimality classification)**.** *For fixed positive B, W, a deterministic online policy optimal on every finite R1 program exists if and only if $W = 1$ or $W \ge B$. The same classification holds for randomized policies required to be optimal in expectation on every input.*

Proof. At $W = 1$, repeatedly issue the oldest ready operation: every nonidle execution takes N cycles. At $W \ge B$, issue every ready operation. The static reduction identifies this with augmented earliest-time layering, which is optimal and needs no future information.

Suppose $B > W \ge 2$ and put $L = \lceil (B-W)/(W-1) \rceil$. The first B operations are roots. An L-job suffix chain starts from a hidden root $x \in \{1, \dots, B-1\}$. These inputs have identical initial views and the same known length $B+L$.

For every x, issue any W roots containing $0, x$ first. Then issue each successive chain job along with up to $W-1$ smallest remaining roots. Before suffix job k, the completed root prefix has length at least $\min(B, 1+(k-1)(W-1)) \ge k$, so admission is legal. The capacity inequality $W + L(W-1) \ge B$ finishes all roots. This realizes $L+1$ cycles and meets the true chain lower bound.

Every deterministic first action either omits 0, delaying suffix admission, or includes 0 and omits some possible x. The corresponding fixed input cannot start its suffix in cycle one, and needs at least $L+2$ cycles. For a randomized first action, average over uniformly chosen x. An action containing 0 hits at most $W-1$ of the $B-1$ choices, and one omitting 0 fails every choice. Positive average failure implies some fixed input with expected duration strictly greater than its optimum. □

The lower-bound family uses only a true chain and isolated vertices. It is an information obstruction, even without a computational complexity bound on the policy. On this family, the exact randomized minimax ratio is $1+(B-W)/((B-1)(L+1))$; choosing 0 plus a uniform $(W-1)$-subset and using one extra root-only cycle after a miss attains it. The complete continuation and advice-counting proofs are in the accompanying theory note.

At $B=3, W=2$, the four-job choices $E=\{1\to 3\}$ and $E=\{2\to 3\}$ force distinct optimal initial batches $\{0,1\}$ and $\{0,2\}$. Every deterministic policy has ratio at least $3/2$ on one. Graham's $2-1/W$ bound applies to every non-delay policy on the augmented DAG, proving the global strict deterministic minimax ratio is exactly $3/2$ at these resources. The global randomized value is only bounded between $5/4$ and $3/2$ here. These ratios allow *no additive constant*; finite witnesses alone do not settle an asymptotic definition that does.

7 Computational and formal verification

The operational Python solver uses completed-set memoization. A separately implemented time-assignment constraint solver shares neither that recurrence nor its state encoding. Operational replay and Bellman checking are also separate implementations. Evidence preserves seeds, source hashes, timings, and exact tested ranges. The following datasets overlap and must not be added as if they were disjoint experiments.

Audit	Actual scope
Root exhaustive baseline	All 1,100 forward DAGs with $N \le 5$; 26,706 resource cases, plus 100 seeded fan-in-two random graphs.
Independent solver agreement	17,378 instances: all 684 fan-in-two DAGs with $N \le 5$ at all tested resources, plus 1,000 seeded larger instances.
Static/operational bridge	264,121 finite time assignments or replay cases on all 75 DAGs with $1 \le N \le 4$.
Fixed-width threshold family	216 horizon-K decisions, $2 \le K \le 10$, all windows, widths two and three; independent witness replay.
Depth-two threshold family	25 programs, 5,720 horizon-two decisions, $2 \le m \le 10$, all allowed r and all windows/widths.
Approximate saturation family	2,020 exact resource comparisons for $1 \le K \le 10$, including oversized windows; 1,185 tolerance thresholds.
Compact certificates	350 independent random instances and 24 deliberately malformed certificates.

Lean 4.34.1 (compiler commit `5045d0056413266e57c625dcd7c365b10e377c52`) uses Std, with no Mathlib dependency. The main modules expose 68 audited declarations; the resource-footprint module adds 18, and three supplementary Bellman/search modules add 20 distinct declarations (106 in total). Root recompilation uses fresh temporary compiled objects. Audits permit only Lean's standard `propext`, `Classical.choice` and `Quot.sound`; no new unproved axiom, `sorry`, or `admit` is used. A documented process-path compatibility shim is a host runtime accommodation and does not change the compiler or kernel.

The formal scope includes operational/time-assignment equivalence, finite optimality, resource caps, the exact fixed-schedule minimum-window formula, a concrete executable time-and-lane enumeration algorithm with soundness/completeness/optimality, operational Bellman lower

certificates, a general attaining-potential theorem, and the arbitrary-local- policy four-job obstruction. Generic finite-search lemmas and the actual concrete enumerator are distinguished. These proofs do not verify the Python program, the optimized frontier DP implementation, the full general online theorem, or the unbounded saturation families. Finite tests are not universal proofs. Independent agent review is not academic peer review.

8 Limitations and future work

The model omits variable operation latency, functional-unit classes, finite retirement bandwidth, physical-register allocation, memory hierarchy, speculation, branches, exceptions, dynamic power and clock constraints. Its online information model is deliberately explicit and may differ from what a concrete processor can infer. No implementation-level speed or energy claim follows from these theorems.

The next questions are robust approximate saturation, tight online ratios at other fixed resources, composable hidden-choice adversaries, efficient certificates at finite issue width, and formalization of the general resource and observation theorems. A deep originality audit remains needed, especially for bounded-lookahead and FIFO storage scheduling. All research states and reproducible commands are saved in `STATUS.md` and `HANDOFF.md`; the authorized research branch is `partner/dist-ILP1-20261014`.

References

[1] D. W. Wall. *Limits of Instruction-Level Parallelism*. WRL Research Report 93/6, 1993. https://davidwall.info/papers/WRL-TR-93.6.pdf.

[2] P. K. Dubey, G. B. Adams III, M. J. Flynn. *Instruction Window Size Trade-Offs and Characterization of Program Parallelism*. IEEE Transactions on Computers, 1994. https://doi.org/10.1109/12.278481. Institutional record inspected; full-text access incomplete.

[3] B. Fields, S. Rubin, R. Bodík. *Focusing Processor Policies via Critical-Path Prediction*. ISCA, 2001. https://www.eecg.utoronto.ca/~moshovos/ACA06/readings/critical.pdf.

[4] D. M. Tullsen, B. Calder. *Computing Along the Critical Path*. UCSD Technical Report, October 1998. https://cseweb.ucsd.edu/~calder/papers/UCSD-CSE98-CPProfile.pdf.

[5] S. Navada, N. K. Choudhary, E. Rotenberg. *Criticality-driven Superscalar Design Space Exploration*. PACT, 2010. https://ericrotenberg.wordpress.ncsu.edu/files/2025/06/paper_PACT-19.pdf.

[6] E. G. Coffman Jr., R. L. Graham. *Optimal Scheduling for Two-Processor Systems*. Acta Informatica 1, 200–213, 1972. https://doi.org/10.1007/BF00288685.

[7] R. L. Graham. *Bounds for Certain Multiprocessing Anomalies*. Bell System Technical Journal 45, 1563–1581, 1966. https://doi.org/10.1002/j.1538-7305.1966.tb01709.x.

[8] OpenAI. *Three-machine unit-time precedence scheduling*, family 124, September 2026 artifact. Inspected repository commit `1581d4cbdd69a0faf34b2089614fe2a814316a6a`. https://github.com/openai/math/blob/1581d4cbdd69a0faf34b2089614fe2a814316a6a/lean/docs/124.md.